

COMP1009

Programming Paradigms

Lecture 001 – Introduction to
objects and Java

This Lecture

- Introduction to the Object Oriented Paradigm
 - The importance of objects
 - Advantages of object oriented paradigm
- Why we will use Java (jobs and simplicity)
- Module Aims and Structure of the OO half
- Introduction to Java
 - By using 'equivalent' C code as an example

The Object Oriented Paradigm

What is the “Object Oriented Paradigm”?

- A style/way of programming
 - A way of thinking about the problem being solved
- Think of the world/program as interacting objects which have a state, and can do things
 - Football myFootball = ...; // An object
 - Person jason = ...; // Another object
 - jason.kick(myFootball); // Do something
- It is ‘more powerful’ than C?
 - **Does not enable** us to do anything more than C
 - In fact basic Java lets us do less (e.g. raw memory/device access)
 - **Does** make some things a lot **easier** to program
 - Hides a lot of complexity (Java hides even more than many)
 - Language or classes do things for you, reducing code writing
 - Makes reuse and modification **a lot easier**

What is an Object Oriented Language?

- A language which promotes or allows object oriented programming and design
- Usual features:
 - Objects (and classes)
 - Encapsulation (methods and data together)
 - Data hiding (restrict access to data)
 - Composition (stronger than aggregation)
 - Inheritance (specialisation, reuse, 'is-a')
 - Abstraction (and interfaces)
 - Polymorphism (dynamic dispatch, late binding)
 - Behaviour only known at runtime

What is an Object Oriented Language

- A language which promotes or allows object oriented programming and design
- Usual features:
 - Objects (and classes)
 - Encapsulation
 - Objects are the key feature
 - ‘things’ which talk to each other
 - Objects work together to solve the problem
 - Data hiding
 - You can think of an object as a collection of data and some ‘functions’ that work on that data
 - Composition
 - Inheritance
 - Abstraction
- Classes are ‘plans’ for an object / types of object, specifying:
 - What is in each object of that type (what data it contains)
 - What object of that type can do (what functions it has)

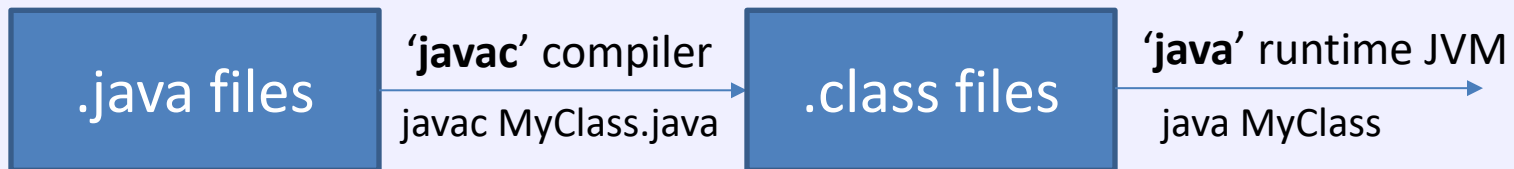
Benefits of OO (why do it)

- Improved understanding of program structure
 - Logical structure, reflecting real world
 - Especially large programs, giving *useful* decomposition
- Ease of (class) re-use
 - Easily re-use, adapt or extend existing work (classes)
- Ease of change
 - Modularity should mean that components/systems can be swapped out more easily (e.g. change implementation)
- Ease debugging
 - Huge problem with data/state – ‘what changed the data?’
 - Easier to enforce constraints (prevent some changes – access limits)
 - Easier to validate changes (add checks)
 - Modular nature and encapsulation help to localise problems
 - i.e. ‘where in the code is the problem?’

Java

The Java Language

- Java has been popular for quite a long time now
- Runs on many devices – without recompiling
- Compiled to an intermediate ‘byte code’ which is executed on target devices

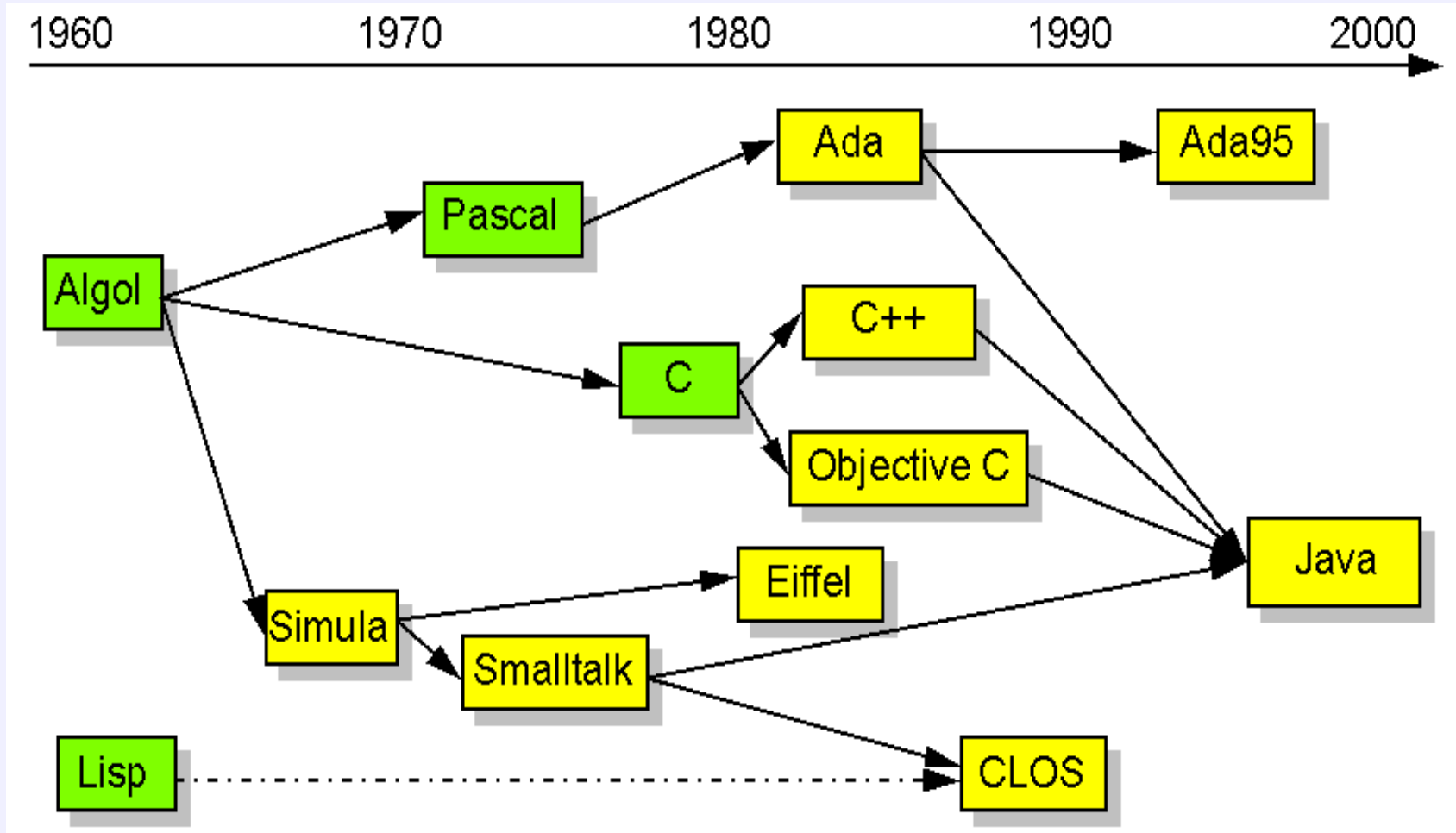


- The resulting .class files can be run on any platform – inside a ‘virtual machine’ (JVM)
- *Compile on one platform, run on any!*

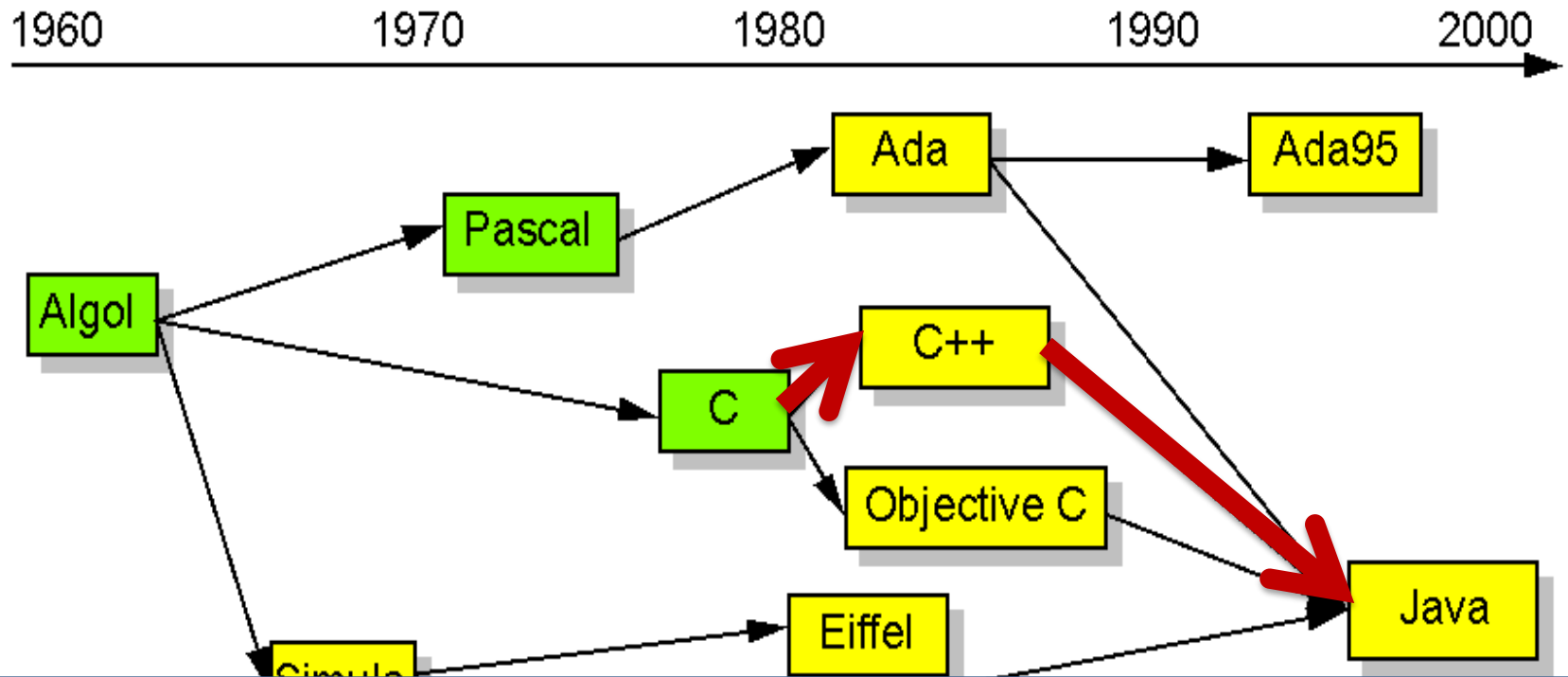
Advantages of Java

- Focus of Java language and library design is *often* on good object oriented design rather than 'speed'
 - E.g. Java Swing library illustrates various patterns
 - C/C++ focus on speed of C/C++ can make 'good OO design' harder to see
- Java has many libraries of reusable classes
 - Classes already exist to do most of what you want to do
- Reuse of classes is important in OO:
 - Speeds up development
 - Speeds up debugging – someone else debugged them
- Aside: What version of Java will we use?
 - We will use Java 21 for assessments, but we will not use many 'recent' features (old books are OK)

Java's Genealogy



Java's Genealogy : C -> C++ -> Java



- It is not just a coincidence that Java is so similar to C
 - Much of the syntax is exactly the same
- Java was designed to be easy for C++ programmers to learn
 - A LOT of C++ programmers at the time – promoting growth
- Changes were deliberate and thought-through, to reduce bugs

Building upon what you know...

- Java is similar to C/C++/C#
 - With important differences and simplifications
- Much that you learned from C is relevant
 - Many syntactic similarities
 - Variables (`int j; float f;`)
 - Conditionals (`if ( j > 1 ) { ... }`)
 - Loops (`for ( j = 0 ; j < 100 ; j++ ) { ... }`)
 - Function calls and returning values (`foo(); return 1;`)
 - You do not need to worry about detail of pointers
 - Java doesn't have these more complex features (C++ does)
- Understanding key Java (OO) ideas will help with other OO languages (particularly C++ & C#)

Building on this with Year 2 C++

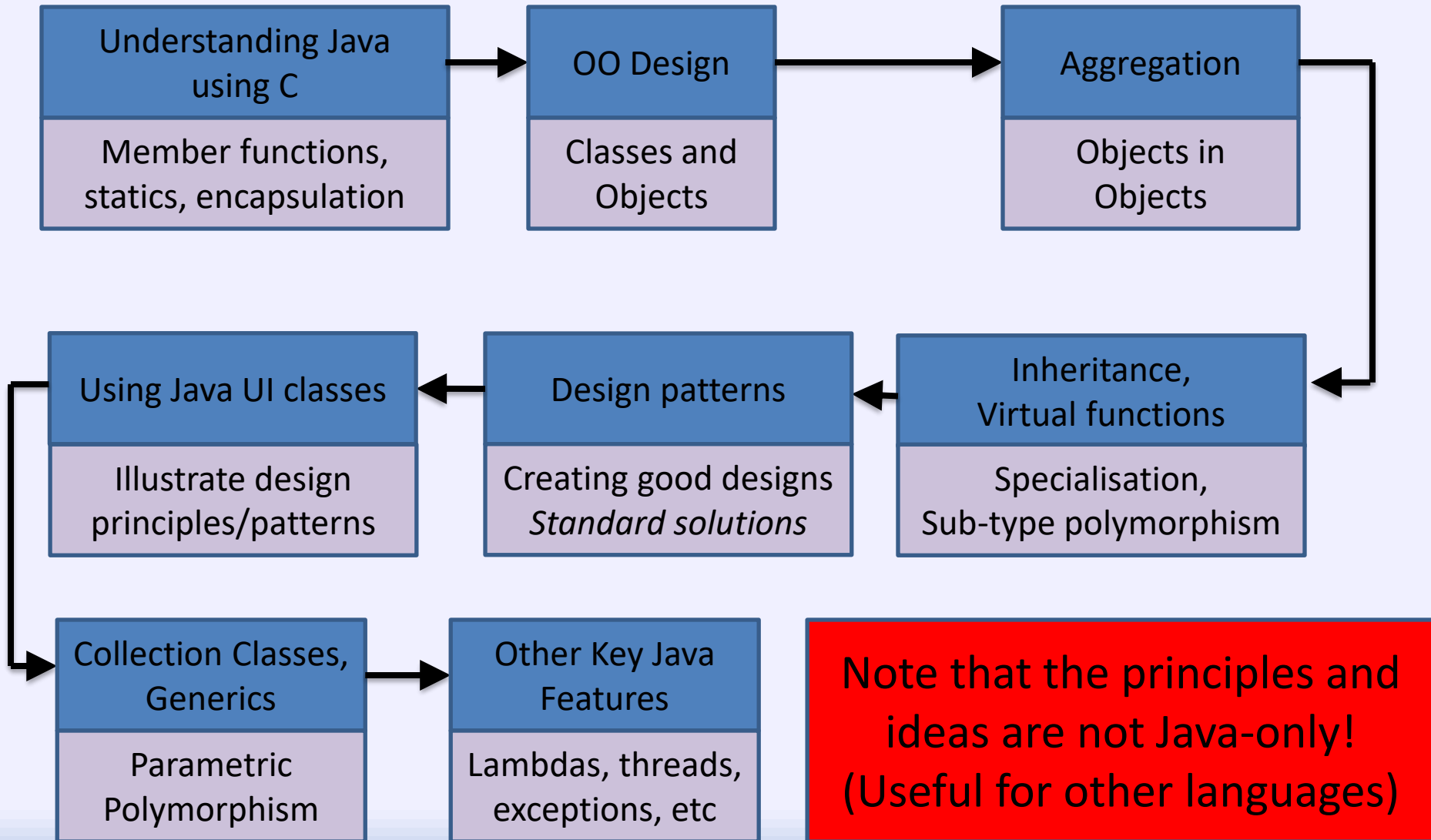
- There should be a C++ optional module next year
 - C++ is much bigger, more complex and more flexible
 - C++ module allows you to build on this and learn more about OO features
 - Java is much easier than C++ to learn
 - If you know Java and C++, languages like C# are easy to learn
- Used in many later modules
 - Java knowledge is assumed
 - Often used by students for group project work
- Good for jobs, and often (still) almost 'expected' of CS grads (a 'proper' programming language)

Module Aims and Structure

Initial aims of Java part of module

- Understand the basics of what Object Oriented means
 - How should we think about the problems?
 - What are the key tools OO provides us with?
- Understand something about Java
 - We can build upon and apply our existing C knowledge
 - Understanding what Java is doing is important for making your code efficient
- **Important note: this is a course on the Object Oriented paradigm, not primarily a course on Java**
 - We won't cover a lot of Java classes – Java has classes for virtually everything – we focus on seeing concepts
 - Sometimes it may be useful to mention other languages
 - We may have a different focus to GCSE / A-level Java

Outline of the Object Oriented side



Assessment

- All Java coursework this year is submitted to GIT
 - A checksubmission process will check it builds/runs to give you peace of mind when I mark it (using same environment)
- Two small Initial Courseworks (1% + 2%)
 - Plus various optional lab exercises that will help you more
- One Main Coursework in two parts (7% + 10%)
 - Simple Java board game (Othello/Reversi)
 - Using some interfaces that I provide to you!
- Exam – Examsys exam (60%)
 - OO/Java questions ask you to show your knowledge of OO concepts using Java
 - E.g. what classes would you use for this? Provide the Java code for this class... How would you implement ... ? What design pattern would you use here, and why? How would you implement that in Java?
 - The coursework and lab exercises prepare you for this

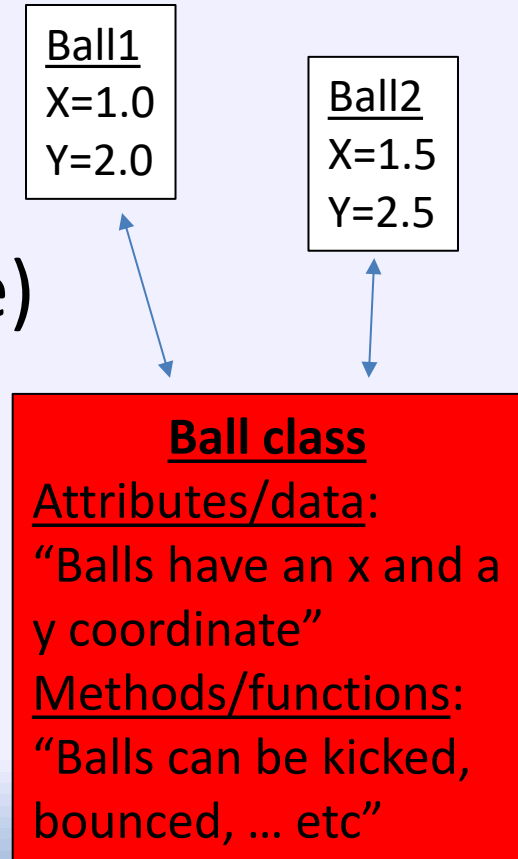
Objects and Classes

Key Concept: Classes and objects

- The concepts of classes and objects are key for the object oriented paradigm
 - Object (actual 'thing')
 - Class (design for things of that type)
- **Classes and objects are different things**
 - Like the difference between the plan a builder used to build a house, and the house itself
 - Or a program you have written, and the process running in memory

Classes vs Objects

- Object (actual 'thing')
 - Thing in your program
 - Some allocated memory which contains some information about the object
 - Has an address in memory
- Class (design for things of that type)
 - Design for an object type
 - What information is stored by the object? (attributes/data)
 - What can the objects do? (what functions can act on them)



Classes and objects in C

Since you know some C:

- structs in C *could* be considered to be very simple objects – without any behaviour (just data)
- You have two different things with structs:
 - A **definition** of a struct in your code
 - Which we could call a **class** in OO terminology
 - **Class** = blueprint for creating an object – I.e. what is in the object, *what can you do to it (in C++ not C)*, etc
 - **Instances** of the struct, in memory at runtime
 - Which we could call **objects**
 - **Object** = some collection of data that exists at runtime, which you can apply functions to (functions listed in class/blueprint)

Consider a C program outline

- Define a structure (struct)
- Create an instance of that struct
 - malloc memory
- Set the values of members of the struct
- Use it, etc
- When finished, free the memory

Don't panic!
You will not need
to write C code!

A **C struct** – define one, create one

```
struct Item // e.g. a Person record
{
    char* stringdata; // e.g. name
    int intdata; // e.g. age
};
```

```
struct Item *pItem = // Create an object
    (struct Item*)malloc( sizeof(struct Item) );
pItem->stringdata = "Jason"; // Name
pItem->intdata = 21; // Age 😊
free(pItem);
```


A **C struct** – a fake class and object

```
struct Item // e.g. a Person record
{
    char* stringdata; // e.g. name
    int intdata; // e.g. age
};
```

class

- struct: Collects data in one place
- Defines the structure: what data does it contain?
- Can create a *struct item*, use it and free it

```
struct Item *pItem = // Create an object
    (struct Item*)malloc( sizeof(struct Item) );
pItem->stringdata = "Jason"; // Name
pItem->intdata = 21; // Age 😊
free(pItem);
```

object

A **Java class** and object

```
public class Item
{
    String stringData;
    int intData;
}
```

class

- **Class:** like a struct (with a lot more features – later)
- Defines the **structure** of the objects
 - And what they can do (later)
- You create objects using 'new' (not malloc())
- No need to free the memory (Java does it for you)

```
Item item = new Item();
item.stringData = "Jason"; // Name
item.intData = 21; // Age 😊
```

object

No sizeof(), no free(), no pointers – much easier? 😊

Classes vs objects : examples

Class (type)	Object (thing)
Person	Jason Any of you
Animal	A specific animal
Tiger [type of animal]	A specific tiger
Car	A specific car
Ford Focus [Type!]	A specific ford focus

- You can have many different ‘instances’ of a class (i.e. objects).
 - e.g. there any many people in the UK, there are many animals.
- Types of things (e.g. the type of animal ‘Tiger’) are ‘classes’, not ‘objects’
 - they are still ‘designs’ (what a Tiger is) rather than a specific Tiger

In Java *all code* is in a class

- We can add 'functions' to classes
- Remember that a class is like a struct in C
 - A grouping of data
 - *But we can also associate functions with that data*
- A class could have no data in fact, just group functions together (more on that later)
- Everything is labelled with access permissions
 - *public* means anything can access it
 - *private* means only the class members can access it
 - (protected and 'package' default will be seen later)

An example class

```
public class DemoClass
{
    private int myInt;

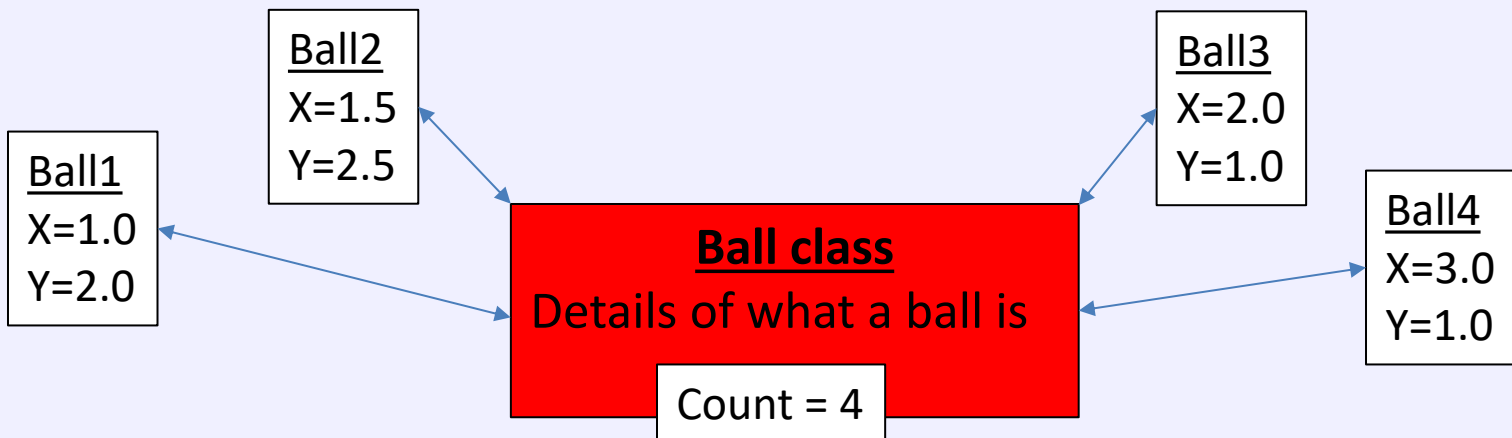
    public int getInt()
    {
        return myInt;
    }

    public void setInt( int newVal )
    {
        myInt = newVal;
    }
}
```

```
DemoClass d
    = new DemoClass();
d.setInt(4);
int j = d.getInt();
```

Static vs normal data and methods

- Normal data and methods are associated with an object
- 'static' data and methods are associated with the class
 - Could be considered as shared between all objects
 - Do not need an object to act upon



- Here, every ball has a position (in each object)
- A count of number of balls could be stored for the class (still needs to be stored in memory)

main() ?

- C programs start with a `main()` function
 - This is a global function in C
 - But Java has no global functions!
 - We have to put it in a class
- `main()` must be **public** – so things outside the class can call it
- `main()` must be **static** because you have no objects when the project starts
- `main()` in java has no return value (**void** type)
- We tell Java which class to run the `main()` for

Simple Java Text Hello World

```
public class HelloWorld
{
    public static void main(String[] args)
    {
        System.out.println( "Hello World!" );
    }
}
```

Compiled with: `javac HelloWorld.java`

Executed with: `java HelloWorld`

Next Lecture

- Next lecture we will look at how to use your C knowledge to start to build Java programs
- Lab 1 is available on the moodle page
 - See section to this lecture
- CW2a is available, but do lab 1 first
- All recordings for OO/Java are available using the link in the OO/Java lecture slides section
- I suggest to use/install Eclipse Java: (free!)
 - <https://eclipseide.org/>
 - Choose “Eclipse IDE for Java Developers”
- Back your work up regularly